

Mr.TPL: A Method for Multi-Pin Net Router in Triple Patterning Lithography

Chengkai Wang¹, Weiqing Ji^{1*}, Mingyang Kou¹, Zhiyang Chen², Fei Li³, Nengyong Zhu³ and Hailong Yao^{1,4*}

¹University of Science and Technology Beijing, Beijing, China

²Tsinghua University, Beijing, China

³Empyrean Technology Co., Ltd., Beijing, China

⁴Key Laboratory of Advanced Materials and Devices for Post-Moore Chips, Ministry of Education, Beijing, China

chengkaiwang@xs.ustb.edu.cn, weiqingji@ustb.edu.cn, kmy@ustb.edu.cn,

chenzhuy21@mails.tsinghua.edu.cn, lifei2@empyrean.com.cn, Zhunyu@empyrean.com.cn, hailongyao@ustb.edu.cn

Abstract—Triple patterning lithography (TPL) has been recognized as one of the most promising solutions to print critical features in advanced technology nodes. A critical challenge within TPL is the effective assignment of the layout to masks. Recently, various layout decomposition methods and TPL-aware routing methods have been proposed to consider TPL. However, these methods typically result in numerous conflicts and stitches, and are mainly designed for 2-pin nets. This paper proposes a multi-pin net routing method in triple patterning lithography, called Mr.TPL. Experimental results demonstrate that Mr.TPL reduces color conflicts by 81.17%, decreases stitches by 76.89%, and achieves up to $5.4\times$ speed improvement compared to the state-of-the-art TPL-aware routing method.

Index Terms—triple patterning aware routing, detail routing, multiple patterning lithography

I. INTRODUCTION

Triple patterning lithography has been recognized as one of the most promising solutions to print critical features in advanced technology nodes [1]. The primary challenge of TPL lies in the efficient decomposition of design layouts, ensuring that patterns are correctly assigned to different masks.

In recent years, various layout decomposition methods [2]–[4] and TPL-aware routing methods [5]–[7] have been proposed to consider TPL. Li et al. developed a layout decomposition method, called OpenMPL [2], which does not change the layout but assigns features that are close to each other to different masks. Zhang et al. proposed a dedicated low-rank SDP algorithm for MPL decomposition with augmented Lagrangian relaxation and Riemannian optimization [3]. However, since the layout patterns remain unchanged, existing layout decomposition methods inevitably lead to unsolvable color conflict issues. As shown in Fig 1(a), the four patterns are too close to each other, making it impossible to assign three colors in a rule-compliant manner, thus introducing an unresolvable color conflict. Fig 1(b) is a reasonable solution.

Ma et al. introduced a TPL aware routing method that modifies the routing grid, splits each vertice into 12 vertices to represent different masks and directions, and then applies routing [5]. Lin et al. proposed a TPL aware routing method,

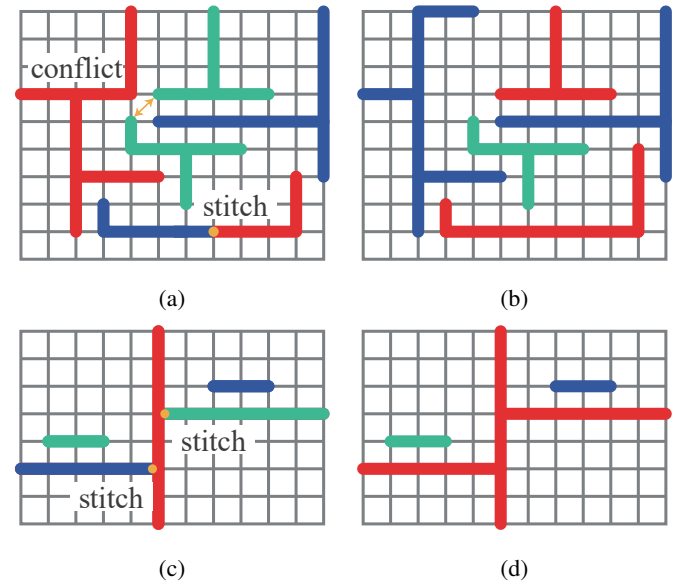


Fig. 1: Examples of TPL solution. (a) The routing solution with an unsolvable color conflict and a stitch. (b)(d) The routing solution without color conflicts or stitches. (c) Stitches caused by 2-pin TPL routing method.

which uses a token graph-embedded conflict graph [6]. On the one hand, most TPL-aware routing methods have a very high time complexity, typically running 3-10 times slower. On the other hand, these methods are primarily designed for 2-pin connections, resulting in an excessive number of stitches when applied to multi-pin nets, as shown in Fig 1(c). This occurs because 2-pin methods cannot dynamically adjust the already-colored paths when connecting multiple pins. Fig 1(d) is a reasonable solution.

To address the above issue, we propose a novel detailed routing method, called Mr.TPL, that considers TPL during the detailed routing phase, specifically targeting multi-pin nets. Our major contributions are as follows:

- We propose a search algorithm based on detail routing that better handles the TPL issues in multi-pin nets. To the best of our knowledge, this is the first work on the routing problem of multi-pin nets in TPL.

*Corresponding author: Weiqing Ji and Hailong Yao. This work was supported in part by the Key Program of National Natural Science Foundation of China (No. 62034005) and the China Postdoctoral Science Foundation under Grant Number 2024M750183.

- We propose a detailed routing method that considering TPL in the routing stage of multi-pin nets, which can greatly reduce the number of color conflicts and stitches.
- We develop a set-based color state merging and vertice coloring method that allows routing path to be in triple color candidate states at the same time during the search process, rather than a determined single color mask.
- Compared with the state-of-the-art TPL-aware routing method [5], Mr.TPL reduces color conflicts by 81.17% and stitches by 76.89%, while achieving a speedup of up to $5.4\times$ on ISPD benchmarks.

The rest of this paper is organized as follows. Section 2 provide the preliminary and problem formulation of the TPL-aware detail routing. Section 3 provides a overall workflow of the proposed method. Section 4 elaborates on the various stages of the method. Section 5 presents experimental results, followed by Section 6, which offers conclusions based on the findings.

II. PRELIMINARY AND PROBLEM FORMULATION

In this section, we first introduce the TPL aware routing. Then, we formulate the TPL aware routing problem for multi-pin nets.

A. TPL-aware Routing

Triple Patterning Lithography (TPL) is a photolithography technique used in semiconductor manufacturing to overcome the limitations of traditional lithography at advanced technology nodes [1]. By splitting a design layer into three separate masks, TPL enables the printing of smaller features, improving resolution. However, TPL introduces significant challenges in mask assignment and routing, as conflicts between features must be avoided to ensure manufacturability. TPL-aware routing integrates mask assignment considerations into the routing stage, addressing mask assignment earlier in the design process.

Fig 1 shows examples of routing solution considering TPL. As shown in Fig 1(a), when the distance between patterns on a layout falls below a predefined threshold, these patterns must be assigned to separate masks to avoid conflicts. If such conflicts arise, one way to resolve them is by assigning the layout to triple masks, introducing a stitch at the boundary where the color changes. Conflicts can lead to printing failures in the metal layout, while stitches may reduce the overall yield. The primary goal of TPL is to minimize both the number of conflicts and stitches. Fig 1(b) shows the routing solution without conflicts or stitches.

B. Problem Formulation

The TPL aware routing problem for multi-pin nets can be formulated as follows.

Given: (1) Layout, including the distribution of pre-placed standard cells, macros, obstacles, and ports. (2) The netlist, which describes the connections between components in the layout. (3) Design rules, including the design rules mentioned in ISPD contests [8] and different mask spacing D_{color} .

Goal: A routed and colored layout that minimizes the weighted sum of conflict cost and stitch cost.

III. OVERVIEW

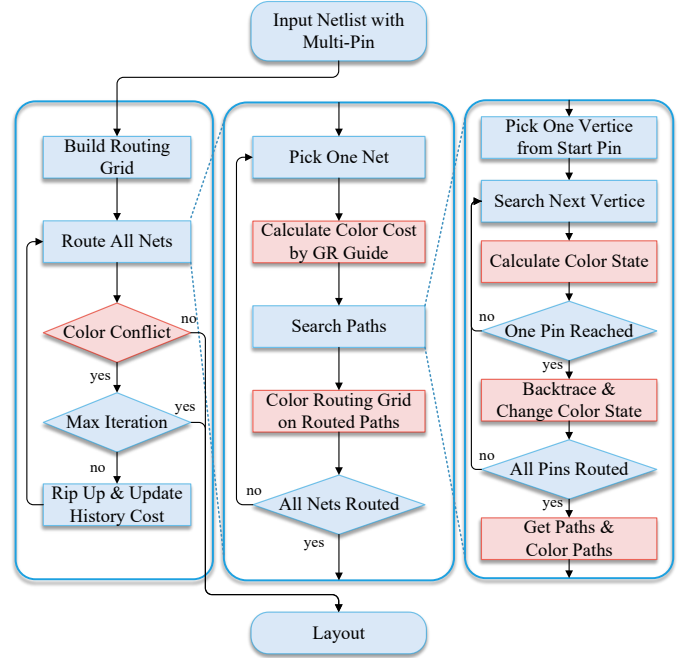


Fig. 2: Overall flow of Mr.TPL

Fig 2 illustrates the overall flow of Mr.TPL. Before routing begins, the necessary data must be fully loaded including layout information, netlist, and global routing results. We also construct the routing grid at this stage. Once the data is ready, the routing process commences. After routing is completed, conflict detection is performed. If conflicts are identified and the maximum iteration limit has not been reached, the process proceeds with rip-up, reroute and updates the history cost accordingly.

The central column details the routing process. At the start of routing, a net is selected for routing. The color cost is calculated with routing regions based on global routing (GR) guide. The path-searching phase follows, during which a colored path is identified. This path is then used to mark the routed vertices and their associated colors on the routing grid. The process iterates until all nets are successfully routed.

The third column describes the detailed steps of the search path process. A pin is randomly selected from the net as the starting point for path searching. Also a vertice is selected within the coverage of the start pin. During searching to the next vertice, the color state of the vertice is calculated. This step is repeated until the next pin is reached. Subsequently, a backtrace phase and color state change is performed. The process repeats until all pins are routed. Finally, the paths are generated and colored.

IV. METHOD

In this section, we will introduce details of Mr.TPL. First, we will introduce the color state and how the routing grid is constructed. Then, we will explain how the cost is calculated when color is introduced. Next, we will describe how the color state of a vertice is determined during the search process

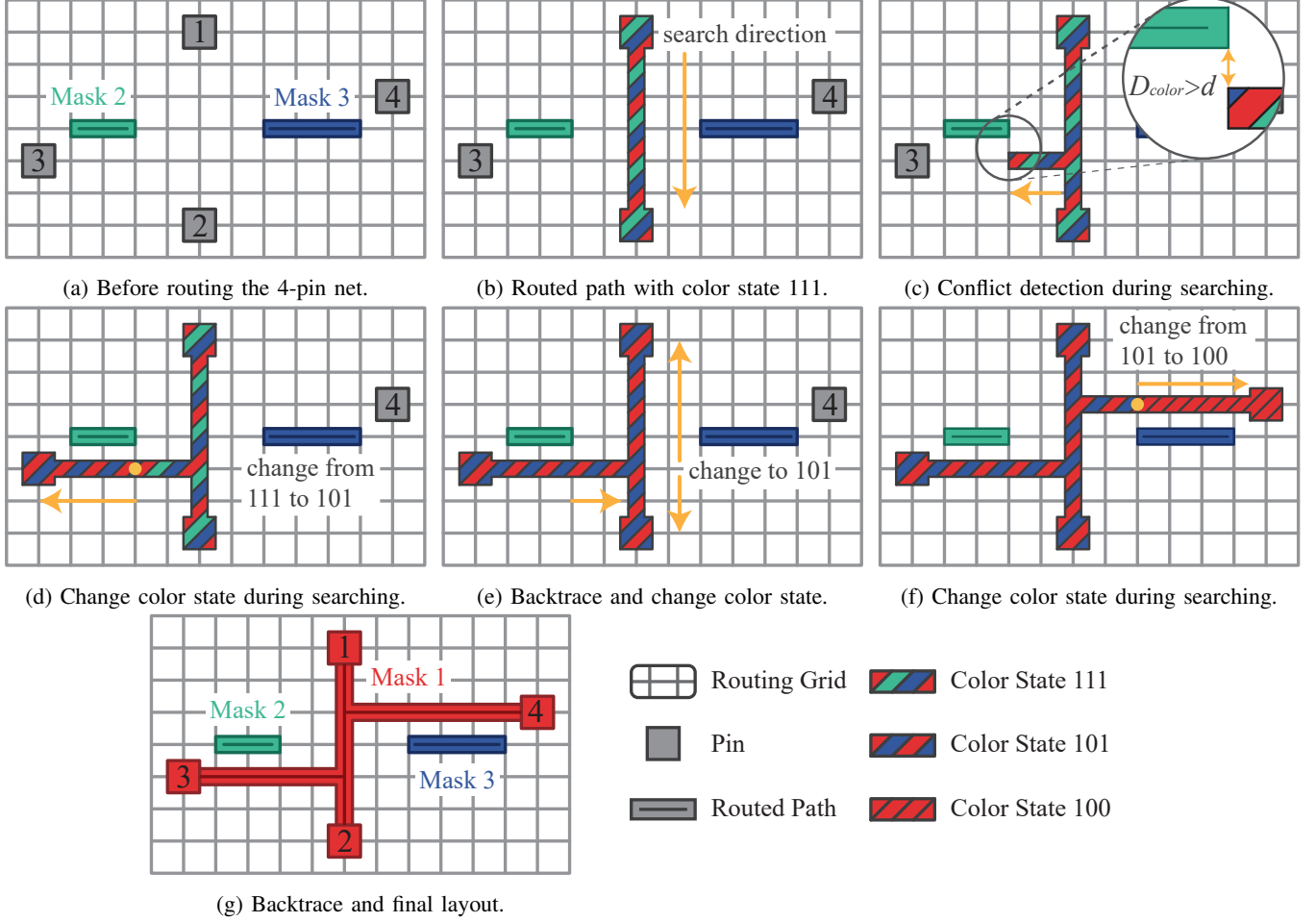


Fig. 3: Example of Mr.TPL.

and how we handle the adding of stitches when necessary. Finally, we will detail our approach to assigning colors to vertices during the backtracing stage. Fig 3 shows an example of Mr.TPL.

A. Color State

To tackle the challenge of color representation in the detailed routing process, we introduce a property called *color state* for each wire segment, which represents a part of the wire. Now we have the definition here.

Definition 1: In Mr.TPL, a *color state* refers to the preparatory assignment of different colors to the routing segments on the same metal layer, ensuring compliance with design rules while minimizing color conflicts and associated manufacturing costs.

This property plays a critical role in managing the triple patterning process and ensuring the correct application of masks. The color state is represented as a 3-bit unsigned integer, effectively encapsulating the necessary information. Each bit corresponds to one of the three masks—red, green, and blue. For example, a binary value of 100 represents red, 010 represents green, and 001 represents blue.

During the routing search process, a wire segment's color state can simultaneously include multiple mask options. When

TABLE I: color state description

Encode	Description
000	none color is allowed
100	only red is allowed
010	only green is allowed
001	only blue is allowed
110	red and green are allowed
101	red and blue are allowed
011	green and blue are allowed
111	all colors are allowed

the color state includes three colors, it implies that assigning any of these colors will incur the same cost. For instance, a color state of 110 indicates that the wire segment can be assigned to either the red or green mask, but not the blue.

At the end of the backtracing phase, a single mask is selected for each wire segment. During this phase, the color state of each segment gradually converges to a single color, ensuring an optimal routing solution. This approach allows for flexible and efficient color management throughout the routing process, significantly enhancing the results in triple patterning lithography.

B. Graph Construction and Cost Calculation

In the initialization phase of triple patterning detailed routing, it is essential to construct the graphical representation of the routing grid. We model it as an undirected graph $G = (V, E)$, where V denotes the set of vertices representing the intersections or corners within the vertice, and E represents the set of edges indicating the connectivity between vertices. To incorporate triple patterning manufacturing considerations, we augment each edge $e \in E$ with an additional attribute, referred to as the *color state*, denoted by $c(e)$. The color state $c(e)$ takes values from a predefined finite set $C = \{c_{100}, c_{010}, \dots, c_{111}\}$, which corresponds to the number of color states used in the process.

The calculation of routing cost is a critical aspect of the detailed routing process. The routing cost for an edge $e \in E$ is influenced by multiple factors, including traditional routing costs, stitch costs, and color conflicts. The overall routing cost $\text{Cost}(e)$ is defined as follows:

$$\text{Cost}(e) = \alpha \cdot \text{Cost}_{\text{trad}}(e) + \beta \cdot \text{Cost}_{\text{stitch}}(e) + \gamma \cdot \text{Cost}_{\text{color}}(e) \quad (1)$$

where:

- $\text{Cost}_{\text{trad}}(e)$ represents the traditional routing cost associated with edge e . The detailed computation is explained in ISPD contest [8], [9].
- $\text{Cost}_{\text{stitch}}(e)$ accounts for the cost associated with stitches, necessary when different segments of a wire are patterned using different masks and must be connected seamlessly.
- $\text{Cost}_{\text{color}}(e)$ reflects the cost associated with color conflicts, which penalizes cases where adjacent edges share the same color state.

In this equation, α , β , and γ are weighting factors that balance the traditional routing costs, stitch costs, and color conflict costs, respectively.

This comprehensive cost function is designed to guide the routing process in an TPL environment, optimize both the physical and manufacturability aspects of circuit design, thereby improving overall performance and reliability.

Algorithm 1: Multi-pin Net Routing

Input: The pin list of the net pins , the routing grid G
Output: Path result resPaths

```

1  $\text{startPin} = \text{pins}[0]$ ;
2  $\text{solQueue} = \{\emptyset\}$ ;
3  $\text{resPaths} = \{\emptyset\}$ ;
4 foreach  $\text{vertice}$  in  $G.\text{get\_covered\_vertices}(\text{startPin})$ 
   do
5    $\text{vertice.cost} = 0$ ;
6    $\text{vertice.colorState} = \text{COLORSTATE}(111)$ ;
7    $\text{solQueue.push}(\text{vertice})$ ;
8 end
9 while not all  $\text{pins}$  routed do
10   $\text{dstVertice} = \text{color\_state\_searching}(\text{solQueue}, G)$ ;
11   $\text{path} = \text{backtrace}(\text{solQueue}, \text{dstVertice})$ ;
12   $\text{resPaths.push}(\text{path})$ ;
13 end
14 return  $\text{resPaths}$ ;
```

C. Multi-pin Net Routing

At this stage, we search for the minimal cost path on the layout, focusing on connecting multiple pins within the net. This method incorporates color conflict costs during the path search, unlike approaches that handle color decomposition after path determination. By considering colors during pathfinding, the solution space is expanded, leading to more optimal results. Algorithm 1 provides the detailed steps.

The process begins with the initial steps (lines 1–3), where the starting pin is determined, typically the first pin in the net's pin-list. An empty priority queue is initialized, using the vertice cost as the priority criterion, alongside a path list to store the routing results. Lines 4–8 involve traversing all vertices of the starting pin, setting their cost to zero, and initializing their color state with 111. These vertices are then added to the solution queue.

Line 9 initiates the search for multi-pin nets by verifying whether any pins remain unrouted. If all pins are connected, the loop terminates. Line 10 performs color state searching, starting from the current vertices in the priority queue (solQueue) and continuing until an unconnected pin is found. Line 11 executes backtracing to finalize the path with its corresponding color state, as discussed in subsequent sections. Finally, Line 12 appends the completed path to the result list.

Algorithm 2: Color State Searching

Input: The priority queue of vertices solQueue , the routing grid G
Output: The vertice of reached pin dstVertice

```

1  $\text{dstVertice} = \text{NULL}$ ;
2 while  $\text{solQueue} \neq \emptyset$  do
3    $\text{vertice} = \text{solQueue.front}()$ ;
4   if  $\text{vertice}$  is covered with an unreached pin then
5      $\text{dstVertice} = \text{vertice}$ ;
6     break;
7   end
8   for  $\text{dir} \in \text{directions} \{F, B, R, L, U, D\}$  do
9      $\text{cost} = [[R, 0], [G, 0], [B, 0]]$ ;
10    for  $\text{color} \in \text{colors} \{R, G, B\}$  do
11       $\text{cost}[\text{color}][0] = \text{color}$ ;
12       $\text{cost}[\text{color}][1] = G.\text{get\_color\_cost}(\text{vertice}, \text{dir}, \text{color})$ ;
13      if  $\text{dir} \notin \{U, D\}$  &&  $\text{color} \notin \text{vertice.colorState}$  then
14         $\text{cost}[\text{color}][1] += \text{stitchCost}$ ;
15      end
16    end
17     $\text{minCost}, \text{colorState} = \text{get\_lowest}(\text{cost})$ ;
18     $\text{newCost} = \text{vertice.cost} + \text{minCost}$ ;
19     $\text{solQueue.update}(\text{vertice}, \text{dir}, \text{newCost}, \text{colorState})$ ;
20  end
21 end
22 return  $\text{dstVertice}$ 
```

D. Color State Searching

The color state searching process is a key step in routing that dynamically evaluates and updates the routing grid to minimize overall cost while considering color conflicts and stitches. This process involves iterative calculations to ensure that paths are

optimized for manufacturability within the constraints of triple patterning lithography. The algorithm takes as input the priority queue and the routing grid, iteratively searching for connections between pins. Algorithm 2 provides a detailed description of this process.

Lines 1–3 initialize the necessary data structures and define the conditions for terminating the search. Lines 4–7 check whether the current vertice is covered by an unreached pin, indicating that a new pin has been located, allowing the search for that pin to end.

Line 8 initiates the search for the next vertice in all possible directions. Line 9 defines a 2-D cost array for the current direction, structured as a 3×2 matrix. The first dimension corresponds to the three possible masks, while the second dimension's first variable stores mask information, and the second variable represents the associated color cost. Lines 10–16 handle the computation and aggregation of color costs: Lines 11–12 calculate the current cost for each mask, while Lines 13–15 evaluate whether a stitch is required and add the associated cost if present. Lines 17–19 then sort the computed color costs to identify the lowest-cost option. This minimum cost is added to vertice.cost and the updated vertice is reinserted into solQueue to continue the search.

Algorithm 3: Backtrace

Input: The priority queue of vertices *solQueue*, the vertice of reached pin *dstVertice*

Output: Path result

```

1 vertice = dstVertice;
2 while vertice ≠ NULL and vertice.cost ≠ 0 do
3   if vertice.verSetPtr = NULL then
4     vertice.verSetPtr =
       make_verSet(vertice.colorState);
5     vertice.verSetPtr.segSetPtr =
       make_segSet(vertice.colorState);
6   end
7   if vertice.prev has common color with vertice
     then
8     if vertice.prev.verSetPtr = NULL then
9       vertice.prev.verSetPtr =
         vertice.verSetPtr;
10    end
11    else
12      tState = get_same_color(vertice.prev,
        vertice);
13      vertice.verSetPtr.segSetPtr.
        change_state(tState);
14      vertice.prev.verSetPtr.segSetPtr =
        vertice.verSetPtr.segSetPtr;
15    end
16  end
17  vertice.cost = 0;
18  solQueue.update(vertice);
19  vertice = vertice.prev;
20 end
21 return get_path(dstVertice);

```

E. Backtrace

In this section, we introduce the backtrace algorithm, which is designed to optimize the tracing and cost evaluation of vertices along routing paths. This algorithm takes as input the

priority queue of the solution and the destination vertice and outputs the finalized routing path. The core concept of the algorithm is to trace backward from the destination vertice to the starting vertice, progressively updating both the cost and color state of each vertice to ensure compliance with design rules and optimal path quality. Before describing the algorithm, it is helpful to introduce two key definitions:

Definition 2: The vertice color-set (*verSet*) is a collection of vertices. These vertices are adjacent on the layout, are searched consecutively, and share the same color state.

Definition 3: The segment color-set (*segSet*) is a collection of verSets. All verSets in this set will have the same color state. Two connected vertices belong to different segSets only if stitch is introduced between them. Otherwise, adjacent vertices must belong to same segSet.

Algorithm 3 begins by setting the current vertice to the destination vertice of the reached pin (line 1). The loop in line 2 continues until the current vertice is either null or its cost is zero. In each iteration (lines 3-6), if the *verSetPtr*, which points to the *verSet* of the vertice, of current vertice is null, a new *verSet* and *segSet* are created based on the color state of the vertice. In lines 7-16, if the predecessor vertice shares the same color state with the current vertice, the algorithm checks if the *verSetPtr* of predecessor is null. If it is, the *verSetPtr* of predecessor is set to the *verSet* of current vertice. Otherwise, the algorithm computes the shared color state of both vertices and modifies the current *segSet* of vertice to reflect this. The *segSetPtr* of predecessor is then set to the current *segSet* of vertice. Throughout each iteration, the update function is called to update the solution of current vertice. Then, the current vertice is updated to its predecessor, and the loop continues. When the loop finishes, the final color of mask is determined by the *dstVertice*, which is the same as the backtracing process in Dijkstra's algorithm.

V. EXPERIMENT RESULTS

We have implemented a triple patterning lithography detailed router in C++ and tested all benchmarks on a Gold 2.30 GHz Intel Xeon Linux Server with 16 cores and 512 GB memory. Mr.TPL is benchmarked against one of the most advanced detailed routing methods for triple patterning lithography [5], as well as the most advanced techniques in multiple patterning lithography layout decomposition [2].

Mr.TPL is designed to be integrated into widely used detailed routers. To ensure a thorough comparison, we incorporated our algorithm into Dr.CU(2.0) [10], one of the most advanced detailed routing systems available. Since the triple patterning lithography routing algorithm described in the [5] is not publicly available, we replicated it within the Dr.CU(2.0) framework to provide a fair comparison. For our experiments, we used datasets from both the ISPD 2018 [8] and ISPD 2019 contests [9], applying our multi-mask routing algorithm.

TABLE II: experiment results compared with [5] on ISPD 2018 contest

case	conflict			stitch			cost			runtime (s)		
	[5]	ours	imp.	[5]	ours	imp.	[5]	ours	imp.	[5]	ours	speedup
test1	0	0	zero ^a	3	0	100.0%	2.9545E+05	2.9203E+05	1.15%	59.93	14.98	4.00×
test2	0	0	zero	20	4	80.00%	4.7127E+06	4.6736E+06	0.83%	605.34	156.76	3.86×
test3	0	0	zero	86	16	81.40%	5.3864E+06	5.2565E+06	2.41%	1932.20	518.25	3.73×
test4 ^b	-	4	-	-	615	-	-	1.5055E+07	-	>24h	2796.72	-
test5	2	0	100.0%	99	23	76.77%	1.6376E+07	1.6361E+07	0.10%	14188.33	1110.10	12.78×
test6	17	1	94.12%	185	30	83.78%	2.1340E+07	2.1314E+07	0.12%	4097.95	886.12	4.62×
test7	21	3	85.71%	354	62	82.49%	3.8536E+07	3.8489E+07	0.12%	14944.13	2272.81	6.58×
test8	42	0	100.0%	368	63	82.88%	3.8715E+07	3.8674E+07	0.11%	12584.58	2143.91	5.87×
test9	20	3	85.00%	335	53	84.18%	3.3054E+07	3.3008E+07	0.14%	5385.06	1335.92	4.03×
test10	352	274	22.16%	2151	1710	20.50%	4.3454E+07	4.3643E+07	-0.44%	20931.53	6498.20	3.22×
avg.	50.44	31.22	81.17%	400.11	217.89	76.89%	2.2430E+07	2.2412E+07	0.51%	8303.23	1659.68	5.41×

^a Color conflicts are all 0, so no comparison is made.

^b The running time of the program exceeds 24h, and no data can be obtained.

TABLE III: experiment results compared with OpenMPL [2] on ISPD 2019 contest

case	conflict			stitch		
	[2]	ours	imp.	[2]	ours	imp.
test1	358	0	100.00%	279	7	97.49%
test2	- ^a	15	-	-	388	-
test3	-	2	-	-	13	-
test4	-	109	-	-	1369	-
test5	595	2	99.66%	303	6	98.02%
test6	297	11	96.30%	558	883	58.24%
test7	-	83	-	-	399	-
test8	5782	1	99.98%	4561	150	96.71%
test9	8966	130	98.55%	6852	234	96.58%
test10	10186	260	97.45%	9433	498	94.72%
avg.	4364	61.30	98.66%	3664.33	394.70	70.88%

^a OpenMPL [2] does not provide the relevant data, the same below.

A. Compared with the TPL-aware Routing Method

The comparison results are presented in Table 2. The *conflict* column sums the same-color and different-color conflicts. Our approach significantly reduces color conflicts, with an average decrease of 81.17%. The *stitch* column indicates the number of stitches generated during multi-mask routing, where our method achieves an average reduction of 76.89%. The *cost* column reflects the detailed routing cost score as calculated in the ISPD 2018 contest, detailed explanations can be found in the contest documentation. Our method also shows an improvement in this metric, with a 0.51% reduction in cost. The final column compares the runtime of the algorithms. As anticipated from our earlier complexity analysis, our approach demonstrates a significant advantage in computational efficiency, achieving a speedup of up to 5.4×. Notably, in the fourth example, *ispd18test4*, the DAC 2012 method failed to produce valid results within 24h, whereas our algorithm successfully completed the task, underscoring the superiority of our approach.

B. Compared with the Layout Decomposition Methods

We conducted a second experiment to compare our algorithm with the most common multiple patterning lithography layout decomposition algorithm, OpenMPL. Using the dataset from the ISPD 2019 contest, we ensured a fair comparison by utiliz-

ing the data presented in the OpenMPL paper [2]. Additionally, the input routed layout is generated by Dr.CU(2.0) [10]. The results are summarized in Table 3.

The *conflict* column represents the number of color conflicts, while the *stitch* column indicates the number of stitches. Our method demonstrates a significant reduction in both metrics, with color conflicts and stitches decreased by 98.66% and 70.88%, respectively. These results strongly highlight the advantages of considering TPL during the detailed routing stage.

VI. CONCLUSIONS

This paper investigates a triple patterning lithography aware detailed routing algorithm Mr.TPL. We propose a unified routing grid construction model, search, and coloring algorithm that effectively addresses TPL challenges in multi-pin nets. Our approach was compared against the state-of-the-art TPL-aware detailed routing and TPL decomposition algorithms. The experimental results demonstrate significant improvements in color conflict reduction, stitch count, cost score, and runtime, which proves the great effectiveness of our method.

REFERENCES

- [1] D. Z. Pan, L. Liebmman, B. Yu, X. Xu, and Y. Lin. Pushing multiple patterning in sub-10nm: Are we ready? In *Proc. of DAC*, pages 1–6, 2015.
- [2] W. Li and *et al.* Openmpl: An open-source layout decomposer. *IEEE TCAD*, 40(11):2331–2344, 2021.
- [3] Y. Zhang and *et al.* Lrsdp: Low-rank sdp for triple patterning lithography layout decomposition. In *Proc. of DAC*, pages 1–6, 2023.
- [4] H. Liu and *et al.* Layout decomposition via boolean satisfiability. In *Proc. of DAC*, pages 1–6, 2023.
- [5] Q. Ma, H. Zhang, and M. D. F. Wong. Triple patterning aware routing and its comparison with double patterning aware routing in 14nm technology. In *Proc. of DAC*, pages 591–596, 2012.
- [6] Y. Lin, B. Yu, D. Z. Pan, and Y. Li. Triad: a triple patterning lithography aware detailed router. In *Proc. of ICCAD*, ICCAD '12, page 123–129, New York, NY, USA, 2012. Association for Computing Machinery.
- [7] P. Hsu and Y. Chang. Non-stitch triple patterning-aware routing based on conflict graph pre-coloring. In *Proc. of ASP-DAC*, pages 390–395, 2015.
- [8] S. Mantik, G. Posser, W. Chow, Y. Ding, and W. Liu. Ispd 2018 initial detailed routing contest and benchmarks. 2018.
- [9] W. Liu and *et al.* Ispd 2019 initial detailed routing contest and benchmark with advanced routing rules. 2019.
- [10] H. Li, G. Chen, B. Jiang, J. Chen, and E. F. Y. Young. Dr. cu 2.0: A scalable detailed routing framework with correct-by-construction design rule satisfaction. In *Proc. of ICCAD*, pages 1–7, 2019.